

1. (10 puntos) Responder las siguientes preguntas de selección múltiple (varias respuestas posibles por pregunta):

1. En el modelado entidad–relación, ¿cuáles de las siguientes afirmaciones son correctas respecto a la relación “Alumno–Carrera” en una universidad asumiendo que un alumno puede cursar varias carreras? (2.5 puntos)

- a) Es una relación tipo-subtipo.
- b) Debe representarse mediante una entidad intermedia (por ej. CarrerasporAlumno) en el modelo conceptual.
- c) Puede resolverse agregando una FK de Carrera en la tabla Alumno.
- d) Puede resolverse agregando una FK de Alumno en la tabla Carrera.
- e) Puede resolverse agregando dos FKs tanto en la tabla Alumno como en la tabla Carrera.

2. Respecto al proceso de normalización, ¿cuáles de las siguientes afirmaciones son verdaderas? (2.5 puntos)

- a) Una tabla está en 1FN si no contiene atributos multivaluados ni grupos repetidos.
- b) Una tabla está en 3FN si no existen dependencias transitivas entre atributos no clave.
- c) La normalización siempre mejora el rendimiento de las consultas analíticas.
- d) La desnormalización puede ser útil en sistemas transaccionales.
- e) Una tabla está en 2FN si todos los atributos no clave dependen de la clave primaria completa.

3. ¿Cuáles de las siguientes afirmaciones describen correctamente el concepto de **integridad referencial** en SQL? (2.5 puntos)

- a) Impide que existan claves foráneas que no tengan correspondencia como clave primaria en la tabla referenciada.
- b) Permite que se elimine una fila de la tabla referenciada, aunque existan registros de otra tabla que lo referencien.
- c) Se implementa exclusivamente mediante índices.
- d) Refuerza la consistencia entre tablas mediante restricciones de clave foránea.
- e) Es equivalente a la restricción de clave primaria

4. Durante el proceso de diseño de datos, se definen los modelos conceptual, lógico y físico. ¿Cuáles de las siguientes afirmaciones son correctas respecto a sus características y diferencias? (2.5 puntos)

- a) El modelo físico define las estructuras implementables, tipos de datos, índices y restricciones en un DBMS específico.
- b) El modelo lógico y el físico son equivalentes: ambos representan implementaciones concretas.
- c) El modelo conceptual describe las entidades y relaciones del negocio sin referencia al motor de base de datos.
- d) El modelo conceptual incluye decisiones de rendimiento e índices para optimizar consultas.
- e) El modelo físico no puede diferir entre distintos motores de base de datos.

3. (30 puntos) DER. Se desea generar un modelo entidad relación para la siguiente necesidad de negocio:

Entidades
Atributos

Se trata de una aplicación de delivery urbano que conecta **comercios**, **repartidores** y **usuarios** finales. La aplicación permite que los usuarios realicen **pedidos** de **productos**, los cuales son preparados por comercios registrados y entregados por repartidores **independientes** o **asociados** a empresas de logística.

↳ **tipo/subtipo**

Los usuarios se registran con un **identificador**, **nombre**, **apellido**, **mail** y **fecha de alta**. Pueden guardar múltiples **direcciones de entrega** y tienen un tipo de **cuenta** “free” o “premium”. Cada usuario puede realizar pedidos a diferentes comercios.

Los comercios se agrupan en **categorías**, como “restaurantes”, “supermercados” o “farmacias”, y dichas categorías pueden tener una estructura jerárquica (**AUTORREFERENCIA**) (por ejemplo, la categoría “restaurantes” puede tener subcategorías como “pizzerías” o “sushi”). Una subcategoría solo puede depender de una única categoría. De cada comercio se guarda un **identificador**, **nombre**, **fecha de alta** y una **descripción**.

Los repartidores se dividen en dos tipos mutuamente excluyentes: los **independientes**, que trabajan por su cuenta, y los **asociados** (**TIPO SUBTIPO**), que pertenecen a **empresas de mensajería**. Todos los repartidores cuentan con un **identificador**, **nombre**, **apellido**, **documento** y **fecha de alta**. Los independientes tienen además un **CBU** y **tipo de vehículo** (moto, bicicleta o auto), mientras que los asociados cuentan con el **nombre** de la **empresa** y un **número de convenio**. Un pedido puede ser entregado por un único repartidor, y cada repartidor puede realizar múltiples entregas. (UNO A MUCHOS)

Cada pedido es realizado por un usuario hacia un único comercio. Del pedido se registra su **identificador**, **fecha** y **hora de creación**, **estado** (pendiente, en preparación, en tránsito, entregado, cancelado), el **total del pedido**, la **tarifa de envío** y la **propina**. Cada pedido está compuesto por uno o varios **productos** del comercio, no pudiendo mezclar productos de distintos comercios. Para cada pedido se registra también quien fue el repartidor. De cada pedido se registra su **pago**, el cual es único por pedido.

De los **productos** se almacena su **identificador**, **nombre**, **precio base** y **estado** (activo o inactivo). Cada producto pertenece a un único **comercio**.

El **pago** de un pedido puede realizarse con diferentes **medios de pago**, que pueden ser tarjeta de crédito, tarjeta de débito o billetera virtual. De cada pago se registra un **identificador**, una **fecha**, un **monto** y un **código** de autorización.

- A) (15 puntos) Realizar el modelo conceptual de este modelo
- B) (15 puntos) Realizar el modelo lógico

4. (30 puntos) Normalización. Se presenta a continuación una entidad Productos comprados por Cliente que se encuentra denormalizada. Se pide llevarla a 3FN.

IdProducto
NombreProducto
PrecioUnitarioProducto
CategoríasProducto (*se repite por cada categoría que tiene asignada el producto).
IdCliente
NombreCliente.
IdLocalidadCliente
NombreLocalidadCliente
Fecha Nacimiento Cliente
FechaCompra

CantidadComprada

- Esta entidad tiene datos de la cantidad de productos comprados por un cliente en una fecha determinada.
 - Un producto puede pertenecer a varias categorías.
 - De cada categoría solo se tiene una descripción.
- a) (5 puntos) Definir la clave primaria mínima de la entidad considerando los atributos existentes.
- b) (20 puntos) Llevar la estructura a 3era forma normal.

3FN							
Compra		Categorías X Producto		Categorías		Localidad	
<u>IdProducto</u>		IdCategoría (FK --> Categorías)		<u>IdCategoría</u>		<u>IdLocalidadCliente</u>	
<u>IdCliente</u>		IdProducto (FK --> Productos)		Categoría		NombreLocalidadCliente	
<u>FechaCompra</u>							
CantidadComprada		Productos		Cientes			
		<u>IdProducto</u>		<u>IdClientes</u>			
		NombreProducto		NombreCliente.			
		PrecioUnitarioProducto		Fecha Nacimiento Cliente			
				IdLocalidadCliente (FK --> Localidad)			

4. (30 puntos) SQL. Se define el siguiente modelo de datos lógico:

- Producto (IdProducto (PK), NombreProducto, PrecioUnitario)
- Item_Pedido(IdPedido, Item, FechaPedido, IdProducto (FK→Producto), Cantidad, PrecioUnitario, Subtotal, PRIMARY KEY(IdPedido, NroItem))

Producto

IdProducto NombreProducto PrecioUnitario

10	Pizza Muzarella	8000
11	Fainá (porción)	2500
12	Empanadas (docena)	12000
20	Leche entera 1L	1500
21	Yerba orgánica 1kg	4500

Item_Pedido

IdPedido	Item	FechaPedido	Id Producto	Cantidad	Precio Unitario	Subtotal
900	1	5/10/2025	10	2	8000	16000
900	2	5/10/2025	11	1	2500	2500
901	1	5/10/2025	20	4	1500	6000
901	2	5/10/2025	23	1	4500	4500
902	1	5/10/2025	10	1	8000	8000
903	1	5/10/2025	21	4	1500	6000

- a. (10 puntos) Indicar el resultado que devuelve el siguiente query:

```
SELECT IdPedido, SUM(Subtotal)
FROM Item_Pedido
GROUP BY IdPedido
HAVING SUM(SubTotal) > 16000
```

IdPedido	SUM(Subtotal)
900	18500

b) (10 puntos) Indicar el resultado que devuelve el siguiente query:

```
SELECT
  i.IdPedido,
  i.Item,
  p.NombreProducto,
  i.Subtotal
FROM Item_Pedido AS i
LEFT JOIN Producto AS p
  ON i.IdProducto = p.IdProducto
WHERE i.Cantidad = 1
ORDER BY i.IdPedido, i.Item;
```

idPedido	Item	NombreProducto	Subtotal
900	2	Faina	2500
901	2	null	4500
902	1	Pizza Muzzarella	8000

c) (10 puntos) Indicar qué errores tiene el siguiente query. Justificar cada error.

```
SELECT
  i.IdPedido,
  i.Item,
  p.NombreProducto,
  SUM(i.Subtotal)
FROM Item_Pedido AS i
INNER JOIN Producto AS p
  ON i.IdProducto = p.IdProducto
WHERE p.NombreProducto = '%Pizza%'
HAVING SUM(i.Subtotal) > 10000
ORDER BY i.IdPedido, i.Item;
```

- Falta el LIKE
- Falta el Group By antes del Having
- Opcional: agregar un alias **AS Total Subtotal** para mayor claridad.